Ders 19 Genişletilmiş Notu: Süreç Senkronizasyonu: Yarış Durumu, Kritik Bölge, Peterson Çözümü, Donanım Desteği, Mutex ve Semafor

Ders 19 Genişletilmiş Notu: Süreç Senkronizasyonu: Yarış Durumu, Kritik Bölge, Peterson Çözümü, Donanım Desteği, Mutex ve Semafor

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Senkronizasyona Neden İhtiyaç Var?
- 2. Eşzamanlı Çalışma ve Üretici-Tüketici Problemi
- 3. Yarış Durumu
- 4. Kritik Bölge Problemi
- 5. İlk Fikir: Kesmeleri Kapatmak
- 6. Sırayla Geçiş: Nazik Bir Çözüm
- 7. Kritik Bölge Çözümünün Üç Şartı
- 8. Çekirdekte Kritik Bölge
- 9. Peterson Çözümü
- 10. Senkronizasyon İçin Donanım Desteği
- 11. Mutex Kilidi ve Spinlock
- 12. Semafor
- 13. Kilitlenme ve Açlık
- Örnek Sorular

1. Senkronizasyona Neden İhtiyaç Var?

Bu derste senkronizasyon (synchronization) konusuna başlandı. Hoca önce senkronizasyona neden ihtiyaç duyduğumuzu sordu. Aslında bu ihtiyacı doğuran durumları daha önce üç yerde konuşmuştuk: çoklu programlamada (multiprogramming), süreçler arası iletişimde (inter-process communication, IPC) ve iş parçacıklarında (thread).

Bu konuların hepsinde aynı noktaya değinmiştik. Birden fazla süreç ya da iş parçacığı arasında paylaşım veya haberleşme yapmak isterseniz, işletim sistemi bunun için mekanizma sunar. Örneğin paylaşımlı bellek (shared memory) üzerinden haberleşecekseniz, işletim sistemi size iki tarafın da erişebildiği bir bellek alanı verir. İş parçacıklarında da paylaşılan alanlar vardı: veri (data) bölümünde, her iş parçacığının ulaşabildiği ortak değişkenler bulunur.

Bu ortak değişkenler üzerinde birden fazla süreç ya da iş parçacığı değişiklik yaparken her zaman bir sorun çıkabilir: veri tutarsız (inconsistent) hâle gelebilir. Tutarsızlığın kaynağı, süreçlerin belirli kritik bölgelerde çalışması ve bu çalışmayı aynı anda yapmaya çalışmasıdır. Bu yüzden önce kritik bölge (critical section) ve yarış durumu (race condition) tanımlanacak.

Hoca bölümün planını şöyle özetledi:

- **Yazılımsal çözümler:** Uygulanabilir, ama günümüzde pek kullanılabilecek çözümler değil. Peterson çözümüne bakılacak.

- **Donanımsal çözümler:** Çözüm donanımdan geliyorsa kullanmak ve uygulamak çok daha etkilidir. Fakat bu çözümler alt seviyede kaldığı için herkesin kullanımına açmak sakıncalıdır.
- **API düzeyindeki araçlar:** İşletim sistemi bu işi geliştiricilere uygulama programlama arayüzleri (API) ile sunmalıdır. Burada mutex kilidi (mutex lock) ve semafor (semaphore) konuşulacak; mutex'teki spinlock sorunu ve semafordaki durum geçişi sorunu dahil, artıları ve eksileri ile.
- **Klasik senkronizasyon problemleri:** Senkronizasyon sorunları yalnız işletim sistemi seviyesinde değil, uygulama geliştirici seviyesinde de yaşanır. Bunlar çoğu zaman birkaç temel probleme indirgenir. Birincisi, birinin kaynağı üretip yerleştirdiği, diğerinin kullandığı üretici-tüketici (producer-consumer) problemidir. İkincisi, bir işin yapılması için farklı tipte ve sayıda kaynağa ihtiyaç duyulan yapılardır; bunlar genelde yemek yiyen filozoflar (dining philosophers) problemiyle tanımlanır.
- **Monitor:** İşi uygulama geliştirici açısından daha da basitleştiren bir üst seviye yapı. Ardından işletim sistemlerinden örnekler gelecek.

2. Eşzamanlı Çalışma ve Üretici-Tüketici Problemi

İşlemleri hep eşzamanlı (concurrent) çalıştırmaya çalışıyoruz. Herhangi bir t anında çalışan işlemi kesiyor, kenara koyuyor, başka bir süreci alıp çalıştırıyoruz. Zaman paylaşımlı (time-sharing) sistemlerde ve çoklu programlamada bunu yapmak zorundayız; çünkü bütün süreçlerin ilerleyebilmesi gerekir. Böyle bir yapıda süreçler veri paylaşıyorsa çeşitli sorunlar ortaya çıkabilir.

2.1 Tamponu bir sayaçla izlemek

Slayttaki örnek, daha önce gördüğümüz üretici-tüketici problemiydi. Tampondaki (buffer) durumu yalnız bir `counter` değişkeniyle izlemeye çalıştığımızı düşünelim. Sistem açıldı, iki süreç ayağa kalktı. İlk anda tamponda bilgi yoktur, bu yüzden `counter` 0'a ayarlanır. Üretici bir şey ürettiğinde sayacı bir artırır, tüketici bir şey kullandığında bir azaltır.

Üretici bir şey üretirken beklemek zorunda kalabilir. Ayrıca zamanlamada round robin ya da öncelik tabanlı (priority based) bir yöntem kullanılıyorsa, kesme (preemption) yüzünden süreç herhangi bir noktada kenara alınabilir. Bu nokta çok "ilginç" bir yer olabilir.

Hoca sıralama üzerinde durdu. Üretici bir şey üretti; önce tampona koyup sonra sayacı artırabilir, ya da önce sayacı artırıp sonra tampona koyabilir. İkincisi sıkıntılıdır: henüz koymadığınız bir şey için sayacı artırmış olursunuz. Bu yüzden önce koyup sonra artırmak daha makuldür. Ama bunda bile sorun vardır: Tampona bir şey yerleştirdiniz, sayacı artırmadan kesildiniz. Tamponda bilgi var, ama karşı taraf bunun farkında değildir.

2.2 Kesme yalnız satırlar arasında olmaz

Kesmeler yalnız programlama dilinde yazdığınız satırların arasında olmak zorunda değildir. Hoca bunun nedenini komut yaşam döngüsünü (instruction life cycle) hatırlatarak sordu. Bir öğrenci getir, çöz, çalıştır (fetch, decode, execute) adımlarını söyledi. Hoca ardından bu adımlar sırasında ya da her komut çalıştırıldığında başka ne yapıldığını sordu (bkz. Örnek Sorular).

Hocanın ipucu şuydu: İşletim sistemi kesme güdümlüdür (interrupt driven). Peki kesmenin geldiği ne zaman ve nasıl kontrol edilir? Öğrencilerden "zamanlayıcı zaman aşımı (timer timeout)" cevabı geldi. Hoca bunun kesmenin kontrol edilmesi değil, yalnız bir kesme örneği olduğunu söyledi. Zamanlayıcı taşma (overflow) ya da zaman aşımı ile kesme üretebilir, zamanlama için bunu kullanırsınız. Soru derste açık kaldı. Hoca konuya daha sonra kesmeleri kapatma çözümünde geri döndü.

2.3 Hangi paylaşım sorunludur?

Üretici sonsuz bir `while (true)` döngüsünde çalışır. Hoca bu döngünün şart olmadığını, yalnız sürekli üretip tampona koymayı göstermek için yazıldığını söyledi. Sunucu gibi bir yapıda gerçekten böyle de çalışabilir. Üretici, `counter` tampon boyutuna (`BUFFER_SIZE`) eşit olduğu sürece bekler. Eşit değilse, ayrı bir indisle tuttuğu tampon gözüne ürettiğini yerleştirir ve

`counter`'ı artırır. Tüketici ise `counter` 0 olduğu sürece bekler. Sıfırdan farklı olunca tampondan bilgiyi çeker, bir sonraki okuyacağı yeri göstermek için kendi indisini günceller ve `counter`'ı azaltır.

```
/* üretici */
while (true) {
    /* next_produced içinde bir öğe üret */
    while (counter == BUFFER_SIZE) ;   /* bekle */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}

/* tüketici */
while (true) {
    while (counter == 0) ;             /* bekle */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
    /* next_consumed içindeki öğeyi tüket */
}
```

Üretici ile tüketicinin paylaştığı iki yapı vardır: tampon alanı ve `counter`. Hoca tampona nasıl erişildiğini sordu. Bir öğrencinin cevabıyla: Üretici her seferinde tampona yazar, tüketici tampondan okur. Alan paylaşılsa da biri yazma, biri okuma yönünde kullandığı için sorun yoktur. Okuyarak bir değişkenin değerini değiştiremezsiniz. Değer 4 ise bir kez okuyunca 4, ikinci kez okuyunca yine 4 alırsınız. Bu işlem eşgüçlüdür (idempotent): her seferinde aynı sonucu üretir. Yazan da tek taraf olduğu için sorun çıkmaz.

Asıl sorun, her iki tarafın da yazdığı `counter` değişkenindedir. Biri artırır, diğeri azaltır.

2.4 `counter++` makine düzeyinde

`counter` bellekte bir alandır. Mimari dersini ve komutları hatırlarsanız, bu değeri artırmak için önce bellekten bir yazmaca (register) almanız, yazmaçta artırmanız, sonra değeri belleğe geri yazmanız gerekir. Azaltma da aynı şekildedir.

```
/* counter++ */                /* counter-- */
register1 = counter;           register2 = counter;
register1 = register1 + 1;     register2 = register2 - 1;
counter = register1;           counter = register2;
```

Slaytta yazmaçların adları karışmasın diye `register1` ve `register2` diye ayrı verilmiştir. Hoca bunların aynı yazmaç bile olabileceğini söyledi: Süreçler arasında geçerken bağlam değiştirme (context switch) yapılır, yazmaçlar zaten kenara alınır ve diğer süreç için yeniden kullanılır. Yani programcı olarak tek adım gördüğünüz artırma ya da azaltma, aşağıda birden fazla adımdan oluşur. Bu adımların arasında bir kesinti, bir geçiş olabilir.

Slayttaki senaryo şöyledir. Başlangıçta `counter = 5`.

- **Adım 1:** Üretici `register1 = counter` yapar, `register1 = 5`.
- **Adım 2:** Üretici `register1 = register1 + 1` yapar, `register1 = 6`.
- **Adım 3:** Değer belleğe yazılmadan üretici kesilir. Tüketici çalışır, `register2 = counter` yapar. Artırma henüz `counter`'a yansımadığı için `register2 = 5` olur.
- **Adım 4:** Tüketici `register2 = register2 - 1` yapar, `register2 = 4`.
- **Adım 5:** Üretici `counter = register1` yazar, `counter = 6`.
- **Adım 6:** Tüketici `counter = register2` yazar, `counter = 4`.

Tüketici yanlış bir değerden başlamıştır. Bir taraf tamponda 4, diğer taraf 6 eleman olduğunu sanır. Bundan sonra "iki sürecin iki yakası bir araya gelmez". Oysa bir artırma ve bir azaltma yapıldığı için doğru değer 5'tir; ne 4 ne 6 olmalıdır.

3. Yarış Durumu

Bu duruma yarış durumu (race condition) denir. Hoca kavramı şöyle açtı. Süreçlerin komutları farklı uzunluktadır, ama bir komut hangi süreçte çalışırsa çalışsın aynı sürede çalışır. Bir atlama (jump) komutu bir süreç için kaç CPU çevrimi sürüyorsa diğer süreç için de o kadar sürer. Yani süreçler bağımsız çalıştırıldığında birinin daha hızlı, diğerinin daha yavaş çalışması söz konusu değildir.

Fakat kesme yaptığımızda, çalışan süreci ne zaman, nerede ve nasıl kenara aldığımıza bağlı olarak farklı sonuçlar çıkmaya başlar. İş bir yarışa döner, "önde gidenin dediği olur". Örnekte son yazan tüketicidir; ipi o göğüsler ve `counter` 4 kalır. Her çalıştırmada kesintinin nerede olacağına bağlı olarak iki tarafın gördüğü ya da ürettiği değerler farklı olur.

4. Kritik Bölge Problemi

Bir ya da daha fazla sürecin paylaşılan bir ortamda yaptığı işlemler farklı sonuçlar çıkmasına yol açıyorsa, o kod alanına kritik bölge (critical section) denir. Sonucu etkileyen kritik bir iş yapıldığı için bu adı alır. Sistemin tutarlı olmasını istiyorsak, kod ne zaman çalıştırılırsa çalıştırılsın aynı sonucu vermelidir. Kritik bölgenin nasıl ele alınacağına dair çözüm üretme işine kritik bölge problemi (critical section problem) denir. Bunun için farklı protokoller tasarlanır: yazılımsal ve donanımsal.

Slayttaki tanım n süreçten oluşan bir sistemi düşünür: $P_0, P_1, \ldots, P_{n-1}$. Süreç sayısı 2 de olabilir, 100 de. Hoca yine web sunucusu örneğini verdi: Çok iş parçacıklı (multi-threaded) bir web sunucusunun 100 iş parçacığı varsa ve hepsi aynı alana yazıp okuyorsa, kritik bölge bu 100 iş parçacığı arasındadır. Üretici-tüketici gibi iki süreçli bir yapıda da olabilir.

> **Hocanın vurgusu:** İşletim sisteminde tek bir kritik bölge yoktur. "İşletim sisteminde bir tane kritik bölge var, onu çözmeye çalışıyoruz" düşüncesi sık yapılan bir yanlış anlamadır.

Kritik bölge, sistemdeki farklı süreç grupları arasında ayrı ayrı ortaya çıkar. Web sunucusu için çalışan süreçlerin kendi aralarında bir kritik bölge problemi olabilir. Sistemin bambaşka bir yerinde çalışan süreçlerin kendi aralarında bir başkası olabilir. Bu süreçlerle web sunucusu arasında da (servis alıp verdikleri, haberleştikleri için) bir başkası olabilir. Kritik bölge, sistemde çalışan süreçlerin yapısına bağlı olarak ortaya çıkan bir problemdir ve sayısı değişir.

4.1 Giriş, çıkış ve kalan bölüm

Çözümler hep aynı kalıpla ifade edilir:

```
do {
    /* giriş bölümü (entry section) */
        /* kritik bölge (critical section) */
    /* çıkış bölümü (exit section) */
        /* kalan bölüm (remainder section) */
} while (true);
```

Hoca bunu bir oda metaforuyla anlattı. Giriş bölümü genelde bir kilit (lock) üzerinden olur: Anahtarla kapıyı açar, girer, kimse girmesin diye kapıyı kilitlersiniz. Odada işinizi yaparsınız. Çıkarken kapıyı açar, çıkar, anahtarı kapının üzerine bırakırsınız; bu çıkış bölümüdür. Geri kalan kod kalan bölümdür (remainder section). Örneklerde kritik bölgenin içinde ne yapıldığının çoğu zaman önemi yoktur. Önemli olan, o bölgenin kritik olduğu ve korunması gerektiğidir.

5. İlk Fikir: Kesmeleri Kapatmak

Hoca baştaki soruya döndü. Kritik bölge sorunu için süreçlerin ortak değişkenlere yazması gerekir. Ama bir şart daha vardır: Bir süreci baştan sona çalıştırıp bitirsem, sonra diğerini baştan sona çalıştırsam aralarında kritik bölge sorunu çıkmaz. Sorun, sürecin çalışırken kenara alınması (preempt edilmesi) ve diğerine geçilmesiyle ortaya çıkar. Hoca bu geçişin neyle sağlandığını sordu. Cevap kesmedir (interrupt): İşletim sistemi kesme güdümlüdür ve süreçler arası geçişi kesmeler üzerinden yaparız.

O hâlde kritik bölgeye girecek bir sürecin kesilmesini istemiyorsak, en basit çözüm kesmeleri kapatmaktır (interrupt disable). Bu yapılabilir, ama kabul edilemez. Çünkü kesmeleri kapattığınızda aslında işletim sistemini devre dışı bırakmış olursunuz.

Bunun kabul edilebilir olduğu bir örnek vardır: mikrodenetleyiciler (microcontroller). Orada kesilmesini istemediğiniz önemli bir iş yaparken kesmeleri kapatırsınız. Ama iki farka dikkat etmek gerekir:

- Mikrodenetleyicide yazdığınız kod bir işletim sistemi tarafından değil, doğrudan cihaz üzerinde çalıştırılır.
- Mikrodenetleyicide çok kullanıcılı (multi-user) bir ortam yoktur; tek kullanıcılı (single user) bir ortam vardır. Büyük ihtimalle kodun tamamını kendiniz yazarsınız. Başkaları yazsa bile birbirinden bağımsız süreçler olarak gerçeklenir.

Modern işletim sistemlerinde ise bir işletim sistemi, onun üzerinde farklı servisleri yerine getiren işlemler ve aynı ya da farklı kullanıcıların farklı süreçleri vardır. Burada kesmeleri kapatırsanız ortalık "darmadağın" olur.

> **Hocanın vurgusu:** Kesmeleri kapatmak, başı ağrıyan adamı başını keserek tedavi etmek gibidir. Bir çözümdür ama tasvip edilen bir yöntem değildir.

Hoca senkronizasyon çözümlerinin hepsini böyle, mimariye ve yapıya göre getirdiği ve götürdüğüyle konuşacaklarını söyledi. Bir diğer örnek spinlock olacaktı.

6. Sırayla Geçiş: Nazik Bir Çözüm

Slaytta iki süreçli basit bir çözüm gösterildi. Hoca bunu "Peterson çözümüne benzer" bir çözüm olarak tanıttı. P_i ve P_j adlı iki süreç vardır; burada P_i'nin akışına bakılır.

```
/* ortak: int turn; */
/* P_i */
while (true) {
    while (turn == j) ;   /* bekle */
    /* kritik bölge */
    turn = j;
    /* kalan bölüm */
}
```

`turn`, j'ye eşit olduğu sürece P_i döngüde döner, bir şey yapmaz. Sıra kendisine gelince döngüden çıkar ve kritik bölgeye girer. Kritik bölgeden çıkarken sırayı karşı tarafa verir. P_j'nin kodunda i ile j yer değiştirir.

Hoca bunu oldukça nazik bir yaklaşım olarak niteledi: Herkes karşısındakine yol verir. Bilgisayar ağları derslerinde gördüğümüz Aloha protokolüne benzetti. Aloha'da gönderecek bir şeyiniz varsa gönderirsiniz. Çakışmayı (contention) azaltmak için belli zaman aralıklarına bölünmüş slotted Aloha'ya geçersiniz. Sonra p olasılıkla gönderen yapıya geçersiniz: $p = 1$ ise her durumda gönderirsiniz, $p = 0{,}1$ ise yüzde 10 olasılıkla. p azaldıkça, yani taraflar daha nazik davrandıkça, daha iyi sonuçlar elde edilir. Hoca bu nezaketin yansımasını sonraki çözümlerde de göreceğimizi söyledi.

> **Ders sonrası not:** Hoca bu çözümü "Peterson çözümüne benzer" diye gösterdi. Yalnız `turn` kullanan bu çözümün adı sırayla geçiştir (strict alternation). Karşılıklı dışlamayı sağlar, ama bir sonraki bölümdeki ilerleme şartını sağlamaz. P_j kritik bölgeye hiç girmek istemese bile, P_i bir kez girip `turn = j` yaptıktan sonra ikinci kez giremez; P_j'nin girip sırayı geri vermesini bekler. Peterson çözümü bu sorunu `flag` dizisiyle giderir.

7. Kritik Bölge Çözümünün Üç Şartı

Bir süreç kümemiz var ve her sürecin kritik bölgeye girmesini bekliyoruz. Ama aynı anda birden fazla süreç kritik bölgede bulunursa, `register` örneğinde gördüğümüz gibi farklı sonuçlar çıkar. Bir çözümün sağlaması gereken üç şart vardır.

7.1 Karşılıklı dışlama (mutual exclusion)

Herkes kritik bölgeye girebilir, ama bir süreç içerideyken diğerleri giremez. Süreçler birbirini karşılıklı olarak dışlar. Örneğin birinci süreç içerideyse sıfırıncı, ikinci, üçüncü ve dördüncü süreç giremez. Oda metaforuyla: Dört kişiyiz, biri anahtarı çevirip içeri girdi ve içeriden kilitledi. O çıkana kadar diğerleri asla içeri girmemeli. Bu sağlanmazsa baştaki sorun başımıza gelir.

7.2 İlerleme (progress)

Süreçleri lüzumsuz yere bekletmemek gerekir. Dört süreçten hiçbiri kritik bölgede değil ve biri girmek istiyor olsun. Bu süreci bekletmemeli, ilerlemesine izin vermelisiniz. Aksi hâlde girmek istediği hâlde giremeyen herkesin ilerlemesini kesmiş olursunuz.

7.3 Sınırlı bekleme (bounded waiting)

İstekleri sonsuza kadar bekletemezsiniz. Hoca yine dört süreçle bir örnek verdi. Süreçler arasında bir öncelik yapısı olsun; öncelik süreç kimliğine, kullanıcıya, zamanlamaya göre ya da kullanıcı tanımlı olabilir. Yüksek öncelikli bir süreç her seferinde kritik bölgeye girmek istesin ve siz her seferinde izin verin. O girer, çıkar; diğerleri kapıda bekler, bir türlü giremez. Kritik bölgede erişmek istedikleri bir kaynak vardır ve o kaynağa erişimi alamazlar.

Kaynağı yemek gibi düşünürseniz bu iş açlığa (starvation) gider: Süreç kaynağı alamadığı için bekleye bekleye sonunda “açlıktan ölür”. Hiçbir şeyi bitiremez, ilerleyemez. Sistemde öncelik mekanizması olsun ya da olmasın, kritik bölgeye girecek süreçler arasındaki bekleme sonsuza gitmeyecek bir düzene oturtulmalı, bekleme süresi sınırlanabilmelidir.

8. Çekirdekte Kritik Bölge

Sistemde kesme (preemption) varsa bu problem mutlaka ortaya çıkar. Peki çekirdek (kernel) tarafında durum nedir? İş parçacıklarında kullanıcı modu ve çekirdek modu iş parçacıklarını eşleme modellerini hatırlayın. Bir süreç ya da iş parçacığı sistem çağrısı yaptığında çekirdek moduna geçilir. Çekirdekte o çağrıyı çalıştıran bir kod parçası vardır ve onun içinde de kritik bölge olabilir.

Burada çekirdeğin kesilebilir (preemptive) olup olmadığına bakılır. Çekirdek kesilemez (non-preemptive) ise, süreç çekirdek modundan çıkana ya da işlemciyi kendi isteğiyle bırakana kadar çalışmaya devam eder. Hoca bu durumda çekirdek modunda kritik bölgeden bahsetmenin bir anlamı olmadığını söyledi. Çekirdek kesilebilir ise kritik bölgeden bahsedilir ve konuşacağımız yöntemler uygulanır.

> **Ders sonrası not:** Hocanın “çekirdek kesilemez ise çekirdek modunda kritik bölgeden bahsetmek mümkün değil” sözü tek işlemcili sistem için doğrudur. Çok işlemcili sistemde iki çekirdek aynı anda çekirdek kodu çalıştırabilir. Bu yüzden kesilemez çekirdekte de paylaşılan çekirdek verisi için kilit gerekir.

9. Peterson Çözümü

Kritik bölge problemi için önerilen çözümlerden biri Peterson çözümüdür (Peterson’s solution). Hoca çözümün belli açılardan çok temiz ve güzel olduğunu, ama iki süreçlik bir çözüm (two-process solution) olduğunu söyledi. Kritik bölgeye girecek süreç sayısı 2 ise çalışır, değilse sorun vardır.

Çözümün bir varsayımı vardır. Bellekten bilgi almak için `load`, belleğe bilgi koymak için `store` komutu kullanılır. Bu komutların atomik olduğu, yani aralarına kesme girmeyeceği kabul edilir.

Çözümde iki ortak değişken vardır:

- `flag[2]`: Hangi sürecin kritik bölgeye girmeye hazır olduğunu gösterir.
- `turn`: Sıranın kimde olduğunu gösterir.

```
/* ortak: int turn; bool flag[2]; */
/* P_i (j = 1 - i) */
```

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j) ;   /* bekle */
    /* kritik bölge */
    flag[i] = false;
    /* kalan bölüm */
}
```

P_i girmeye hazır olduğunda kendi bayrağını kaldırır (`flag[i] = true`). Burada da aynı nezaket vardır: Süreç i olduğu hâlde sırayı j'ye verir. Sonra bakar: P_j girmeye hazır mı, yani kendi bayrağını kaldırmış mı? Sırayı j'ye vermiştim, sıra hâlâ j'de mi? Hem `flag[j]` doğru hem de `turn == j` ise içeri P_j girmeli, P_i beklemelidir. Bekleme döngüsüne kadar olan kısım giriş bölümüdür. Sonra kritik bölgede ne yapılacaksa yapılır.

Diyelim ki P_j kritik bölgeye girdi. Çıkarken kendi bayrağını `false` yapar. Bu durumda P_i'nin bekleme koşulu bozulur ve P_i kritik bölgeye girer. P_i de çıkarken kendi isteğini `false` yapar ve kalan işine devam eder. Kritik bölgeye yeniden girmek için tekrar hazır olduğunu belirtmesi gerekir. Yani bu da nezaket temelli bir çözümdür ve süreç sayısı iki olduğu sürece gayet güzel çalışır.

9.1 Neden iki süreçle sınırlı?

Hoca süreç sayısı artınca sistemin neden çalışmayacağını sordu ve kendisi açıkladı. İki süreç için bir `turn` değişkeni ve iki elemanlı bir `flag` dizisi yeter. Süreç sayısı artarsa bekleme döngüsündeki kontrollerin de artması gerekir: `flag[0]`, `flag[1]`, `flag[2]`, ... Örneğin 6 süreç varsa her süreç geri kalan 5 sürecin bayrağını kontrol etmelidir.

Fakat kesmenin nerede geleceğini bilemeyiz. Bir kontrolü yaptınız, diğer kontrolleri yaparken kesildiniz. O sırada ilk kontrolün sonucu değişti, ama siz kaldığınız yerden devam ettiğiniz için eski sonuca göre davranırsınız. Hocanın deyişiyle "iş patladı, çatladı, yan yattı". Bu yüzden mümkünse donanım seviyesinde destek veren yapılara ihtiyaç vardır.

> **Ders sonrası not:** Peterson çözümü modern mimarilerde ek önlem olmadan güvenli değildir. İşlemci ve derleyici bellek işlemlerinin sırasını değiştirebilir. Çözümün doğru çalışması için bellek bariyeri (memory barrier) gerekir; ders kitabı da bunu belirtir. Bu not hazırlanırken yukarıdaki kod iki iş parçacığıyla çok çekirdekli bir makinede `gcc -Wall -pthread` ile denendi. Bariyersiz sürümde kritik bölgedeki sayaç birkaç bin eksik çıktı, yani karşılıklı dışlama bozuldu. `flag`/`turn` yazımından sonra bariyer eklenince sonuç her denemede doğru çıktı.

10. Senkronizasyon İçin Donanım Desteği

Aradan sonra hoca, birçok sistemin senkronizasyon için donanım desteği sunduğunu söyledi. Bu desteğin tamamı kilitleme (locking) mekanizması üzerine kuruludur: Kritik bölgeye giderken bir kilidi kilitlersiniz. Kilit kilitli olduğu sürece başka kimse içeri giremez. Temel teknik budur.

10.1 Tek işlemcide kesmeleri kapatmak yeterli mi?

Tek işlemcili (uniprocessor) sistemlerde, mikrodenetleyici örneğinde olduğu gibi, kesmeleri kapatabilirsiniz. Kesilemez bir çekirdeğiniz varsa bunu yaparsınız. Ama daha önce konuşulan sebeplerden dolayı bu güzel bir çözüm değildir.

Ayrıca günümüzde tek bir işlemcimiz olsa bile çoğu zaman içinde birden fazla çekirdek (multicore) vardır. Çok işlemcili (multiprocessor) ya da çok çekirdekli sistemde kesmeleri kapatacaksanız, bunu tüm çekirdeklere ya da tüm işlemcilere ayrı ayrı iletmeniz gerekir. Ayrı ayrı iletmek, araya yine bir gecikme (delay) girmesi demektir. Bu açıdan bakıldığında bu yaklaşım o kadar etkili değildir.

10.2 Atomik komutlar

Bunun yerine atomik komutlar (atomic instructions) oluşturulmuştur. Atomik, araya kesme girmeyen yapı demektir. Bu bir test et ve ayarla (test-and-set) komutu olabilir. Ya da bir bellek bölgesinin değerini başka bir bellek bölgesininkiyle değiştiren bir takas (swap) komutu olabilir.

Bu komutlar mimariden mimariye farklı adlar taşır; doğrudan "test and set" adıyla görmezsiniz. Hocanın örnekleri: 8086'da exchange (`XCHG`) komutu, PowerPC'de ise karşılaştır ve takas et (compare and swap) biçiminde komutlar.

> **Ders sonrası not:** PowerPC atomik işlemleri `lwarx/stwcx.` komut çiftiyle (load-linked/store-conditional) yapar. Doğrudan "karşılaştır ve takas et" biçimindeki komut x86'daki `CMPXCHG`'dir. x86'da `XCHG` bellek işleneniyle kullanıldığında atomiktir.

Mantık hep aynıdır: Kritik bölgeye girerken kilidi ele geçirirsiniz, çıkarken kilidi bırakırsınız ki başkaları girebilsin.

10.3 `test_and_set`

`test_and_set` bir fonksiyon olarak ifade edilebilir. Hoca bunun soyutlanmış bir gösterim olduğunu, fonksiyonun tamamının kesintisiz (atomik) gerçeklendiğini düşünmek gerektiğini vurguladı.

```
bool test_and_set(bool *target) {   /* atomik olarak çalışır */
    bool rv = *target;
    *target = true;
    return rv;
}
```

`target` bir adrestir. Fonksiyon bu adresin içeriğini `true` yapar, öte yandan eski değeri döndürür. Hoca bunun ne işe yarayacağını sordu. Öğrencilerden gelen cevapla: Eski değerin ne olduğunu görebiliriz. Bunun üzerine bir kilidi alma (acquire) ve bırakma (release) mekanizması kurulur. Kilit `false` ile başlatılır:

```
/* ortak: bool lock = false; */
while (true) {
    while (test_and_set(&lock)) ;   /* bekle */
    /* kritik bölge */
    lock = false;
    /* kalan bölüm */
}
```

Süreç kilidi ayarlamaya çalışır. Başkası tutuyorsa `test_and_set` `true` döndürür ve süreç döngüde dönüp durur. Dönen değer `false` ise kilit boştu; artık bu sürecindir, içeri girer ve işini yapar.

> **Ders sonrası not:** Hoca burada bir an "true'ya set ettik, pardon false olarak kaldı" dedi. Doğrusu: `test_and_set` kilidi her durumda `true` yapar ve eski değeri döndürür. Döndürülen eski değer `false` ise kilit alınmıştır ve döngüden çıkılır. Bu not hazırlanırken GCC'nin `__atomic_test_and_set` yerleşik fonksiyonuyla denendi: Boş kilitte ilk çağrı `false`, ikinci çağrı `true` döndürdü; kilidin son değeri `true` oldu.

Önceki yazılımsal çözümlere göre farkı şudur: Bu adımlar atomiktir, bölünmeden çalışır. Birden fazla süreç aynı anda çalışsa da istenmeyen bir değerin oluşmasının önüne geçilir.

10.4 `compare_and_swap`

Mimarilerde başka adlarla da karşılaşabilirsiniz, ama yaptıkları iş benzerdir: Ya `test_and_set`'i ya da `compare_and_swap`'ı farklı bir adla gerçeklerler.

```
int compare_and_swap(int *value, int expected, int new_value) {   /* atomik */
    int temp = *value;
    if (*value == expected)
        *value = new_value;
```

```
    return temp;
}
```

Üç parametre vardır. `value` değiştirmeye çalıştığımız bellek bölgesidir. `expected`, adı üstünde, olmasını beklediğimiz değerdir. `new_value` ayarlamaya çalıştığımız yeni değerdir. Fonksiyon, değerin beklenene eşit olup olmadığına bakar; eşitse yeni değere ayarlar. Bu atomik yapıldığı için değişiklik yalnız `value` `expected`'a eşitken gerçekleşir. Böylece yarış durumundan kaynaklanan sıkıntılardan arınmış oluruz.

10.5 Sınırlı bekleme eklemek işi karmaşıklaştırır

Hoca Peterson çözümünü hatırlattı: Orada girme niyetimizi (`flag`) ve sıranın kimde olduğunu (`turn`) biliyorduk. `test_and_set` ile karşılıklı dışlamayı uygulayıp bekleme süresini de sınırlamak (bounded waiting) isterseniz, slaytta görüldüğü gibi kod çok daha karmaşık hâle gelir. Karşılıklı dışlama olmazsa olmazdır, ama sınırlı bekleme gibi bir kısıt eklendikçe iş karmaşıklaşır. Bu, işletim sisteminin bu yapıyı ele almak için daha fazla zaman, emek ve kaynak harcaması demektir. Bu yüzden çözümler mümkün olduğunca basite indirgenerek kullanılır; bunun nasıl yapıldığı mutex kilidinde görülecekti.

> **Ek bilgi (derste anlatılmadı):** Ders kitabındaki sınırlı beklemeli `test_and_set` algoritması, `lock` değişkenine ek olarak `bool waiting[n]` dizisi kullanır. Süreç `waiting[i] = true` yapar ve ya `waiting[i]` `false` olana ya da `test_and_set(&lock)` `false` döndürene kadar bekler. Kritik bölgeden çıkan süreç, $i+1, i+2, \ldots$ sırasıyla dolaşarak bekleyen ilk süreci arar. Bulursa kilidi bırakmadan o sürecin `waiting` değerini `false` yapar, yani kilidi doğrudan ona devreder; bulamazsa `lock = false` yapar. Böylece her süreç en fazla $n-1$ tur bekler.

10.6 Donanım çözümleri geliştiriciye doğrudan açılmaz

Buraya kadarki çözümler donanım seviyesindedir ve belli açılardan karmaşıktır. Donanım seviyesinde oldukları için uygulama geliştiricinin doğrudan kullanımına da açık değildir. İşletim sistemi, süreçlerin bu işi yapabilmesi için mekanizmalar üretmelidir. Bunlardan biri mutex kilididir.

11. Mutex Kilidi ve Spinlock

Mutex kilidi (mutex lock), arka planda donanımın sağladığı atomik işlemleri kullanan, ama bize yalnız değerinin değiştirildiği bir yapı olarak görünür. Boolean olarak bakarsanız değeri `true` ya da `false` yapılır. İki çağrısı vardır:

- `acquire()`: Kilidi ele geçirmeye çalışır.
- `release()`: Kilidi diğerleri kullanabilsin diye bırakır.

```
acquire() {
    while (!available) ;   /* meşgul bekleme */
    available = false;
}

release() {
    available = true;
}

/* kullanım */
while (true) {
    acquire();
    /* kritik bölge */
    release();
    /* kalan bölüm */
}
```

11.1 Spinlock ve meşgul bekleme

Mutex'i gerçeklerken işletim sistemi olarak farklı yaklaşımlar seçebilirsiniz. Yukarıdaki biçimde `acquire`, kilidin müsait olup olmadığına bir `while` döngüsü içinde sürekli, dönüp dönüp bakar. Kilit başkası tarafından bırakılmadan alamazsınız, ama alamadığınız süre boyunca döngüde kontrol edersiniz. Bu yüzden bu yapıya spinlock denir.

Hoca bunu çizgi filmlerdeki (örneğin Simpsonlar'daki) arabada sürekli "geldik mi, geldik mi?" diye babasının başının etini yiyen çocuğa benzetti. Normalde kaynağı bekleyen bir süreç bekleme durumunda olmalıdır. Spinlock ise elinde olmayan kaynağın olup olmadığını sürekli sorar. Yani meşgul bekleme (busy waiting) yapar: Bir iş yapıyordur, ama yaptığı iş yalnız kaynağın müsait olup olmadığını sormaktır. Bu da işlemci kaynağının boşuna kullanılmasına yol açar.

11.2 Alternatif: uyutmak mı, bekleme kuyruğu mu?

Hoca alternatifin ne olabileceğini sordu (bkz. Örnek Sorular). Öğrencilerden süreci uyutmak (`sleep`) önerisi geldi. Hoca bunun mantıklı olmadığını söyledi: Uyutmak yalnız meşgul beklemenin süresini azaltır, kendisini ortadan kaldırmaz. Çünkü kaynağın ne zaman müsait olacağını bilmezsiniz. 5 saniye sonra müsait olacak bir kaynak için süreci 1 saniye uyutursanız, kalan 4 saniye yine meşgul bekleme yaparsınız. 1 saniye sonra müsait olacak bir kaynak için 5 saniye uyutursanız, süreci 4 saniye boşuna uyutmuş olursunuz. Uyutmak ya sorunu erteler ya da daha beter hâle getirir.

Doğru yaklaşım, G/Ç (I/O) işlemlerinde yaptığımızın aynısıdır. Bir süreç G/Ç başlattığında bekleme kuyruğuna (wait queue) gider; G/Ç bitince ona haber verilir. Aynı mantıkla, süreç kilidi almak istediğinde onu bekleme kuyruğuna koyarız. Biri kilidi bıraktığında, yani kilit müsait olduğunda, bu süreci çalıştırılmak üzere tekrar hazır kuyruğuna (ready queue) alırız.

Bu güzel bir yapıdır, ama sıkıntısı vardır. Her seferinde zamanlayıcının (scheduler) yapıları içinden bir daha geçersiniz. Ek listeler tutmanız gerekir: Bu kilidi bekleyen süreçler kimler, hangisini nereye koyacağım? İş bir adım daha karmaşıklaşır.

11.3 Hangisi ne zaman tercih edilir?

- **Tek işlemcide:** İşlemciyi boşuna meşgul etmek yerine süreci bekleme kuyruğuna almak daha makuldür. Bağlam değiştirme yapılır, ama boşuna çalışan işlemci kurtarılır ve başka işler çalıştırılır.
- **Çok işlemcide:** Hoca bunun ilginç geleceğini söyledi. Kilit için bekleme süresi uzun değilse, bağlam değiştirme yerine spinlock tercih edilir. Bir işlemci spinlock'ta beklerken diğer işlemcide başka bir iş çalışır. Kilidi tutan iş kısaysa, bir süre sonra kilit müsait olur ve bekleyen işlemci de devam eder. Boşu boşuna bağlam değiştirme yapılmaz.

  Hocanın vurgusu: Spinlock'u tercih etmek için iki şart vardır: birden fazla işlemci olmalı ve kilitlenen iş hızlı, kısa bir işlem olmalı. Aksi hâlde spinlock makul değildir.

12. Semafor

Bekleme kuyruğu ile anlatılan bu yapı kullanılır ve adı semafordur (semaphore). Hoca adın kökenini anlattı. Yapıyı ortaya atan kişi onu tren raylarından esinlenerek tasarlamıştır. Semafor aslında bayrak demektir; tren yollarındaki geçişlerde kullanılan işaret bayrağıdır.

Hoca İstanbul'dan bir örnek verdi: Kabataş tramvay hattında perondan çıkarken iki yol vardır. Çıktıktan hemen sonra bir makasa girer, tek yola inersiniz; sonra tekrar makaslanıp çift yola çıkarsınız. Uzun şehirlerarası demiryollarında da bazı köprüleri ya da noktaları geçmek için, maliyeti azaltmak ya da başka kısıtlar yüzünden, birden fazla ray yerine tek ray döşenir. Bu tek raya geçişlerde trenler eskiden birbirlerine bayrak sallayarak haber verirdi; bugün başka sinyal mekanizmaları kullanılıyor. Semafor da bu işi yapar.

> **Ek bilgi (derste anlatılmadı):** Semafor kavramını Hollandalı bilgisayar bilimci Edsger W. Dijkstra büyük olasılıkla 1960'ların ortasında ortaya atmıştır. Dijkstra işlemlere Hollandaca kelimelerden gelen P ve V adlarını vermişti; ders kitabı bunları `wait()` ve `signal()` olarak adlandırır.

12.1 Tanım

Semafor bir tam sayı (integer) değişkendir. Üzerinde iki işlem yapılır: `wait()` ve `signal()`.

- `wait()`: Tam sayı değeri sıfıra ya da negatife iniyorsa çağrı bloklanır, çağıran süreç bekletilir.
- `signal()`: Değişkenin değerini artırır.

İki tür semafor vardır:

- **İkili semafor (binary semaphore):** Değeri 0 ile 1 arasında değişir. Hoca bunun, arkadaki gerçekleme olarak değil, kullanım mantığı olarak mutex ile aynı olduğunu söyledi. Mutex'te `acquire` ve `release` vardı, burada `wait` ve `signal` var.
- **Sayan semafor (counting semaphore):** Kaynağınız tek değil, birden fazlaysa kullanılır. Örneğin 5 eş kaynağınız ve bunları kullanmak isteyen 8, 10 ya da 100 süreciniz var. 5 kaynağı birden bir sayan semaforun arkasına koyarak koruyabilirsiniz.

Hoca bir uyarı yaptı: İkili semaforu kaynak korumak için kullanıyorsanız ve 0 ile başlatırsanız, kaynağı kullanmak için `wait`'i öne koyduğunuzda hiçbir süreç ilerleyemez. Buna dikkat etmek gerekir.

12.2 Semaforla sıralama

Slayttaki örnekte P_1 ve P_2 süreçleri vardır. P_1'deki S_1 deyiminin P_2'deki S_2 deyiminden önce çalışması isteniyor. Bunun için `synch` adlı bir semafor oluşturulur ve 0 ile başlatılır.

```
/* ortak: semaphore synch = 0; */
/* P1 */                /* P2 */
S1;                     wait(synch);
signal(synch);          S2;
```

Hoca bu yapının özel olduğunu vurguladı: Ortak bir kaynağı kullanmak için değil, süreçlerin sırasını sağlamak için kurulmuştur. Bu yüzden az önceki uyarının tam tersi yapılır, semafor 0 ile başlatılır. P_2 `wait(synch)` noktasına geldiğinde değer 0 olduğu için otomatik olarak bloklanır. P_1, S_1'i tamamlayıp `signal(synch)` yapınca P_2 bloktan kurtulur ve hayatına devam eder.

> **Ders sonrası not:** Bu not hazırlanırken örnek POSIX semaforlarıyla (`sem_init`, `sem_wait`, `sem_post`) derlenip çalıştırıldı. P_2 önce başlatılıp P_1 bir saniye geciktirildiği hâlde S_1 her zaman S_2'den önce çalıştı.

12.3 Bekleme kuyruğuyla gerçekleme

Semaforda `wait` yapan bir sürecin bekletilmesi gerekiyorsa, süreç bekleme durumuna (wait state) gider. Başka bir süreç `signal` işlemini yapana kadar orada kalır. `signal` ile bekleyen süreç bekleme durumundan alınıp hazır duruma konur, yani hazır kuyruğuna eklenir.

Bunun için her semaforun veri yapısında, o semaforu bekleyen süreçlerin listesi tutulmalıdır. Her zaman olduğu gibi bunun için bağlı liste (linked list) kullanılır.

```
typedef struct {
    int value;
    struct process *list;   /* bu semaforu bekleyen süreçler */
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        /* bu süreci S->list'e ekle */
        block();
    }
}

signal(semaphore *S) {
    S->value++;
```

```
    if (S->value <= 0) {
        /* S->list'ten bir P süreci çıkar */
        wakeup(P);
    }
}
```

> **Ders sonrası not:** Hoca `signal` işleminin o semaforu bekleyen "tüm süreçleri" hazır kuyruğuna aldığını söyledi. Kitaptaki tanımda `signal` yalnız bir süreci uyandırır (`wakeup(P)`). Bütün bekleyenleri uyandırmak ayrı bir işlemdir (örneğin koşul değişkenlerinde `broadcast`). Bu gerçeklemede değer negatif olabilir; negatif değerin mutlak değeri bekleyen süreç sayısını verir.

Süreci bekleme durumuna alıp sonra hazır kuyruğuna geri koymak etkili görünür. Ama özellikle çok işlemcili bir yapıda gerçeklemeye kalktığınızda ortalık daha da karışır. Bu karmaşıklık yüzünden, kilidin kısa süre tutulacağını bildiğiniz durumda işi spinlock olarak gerçeklemek çoğu zaman daha etkilidir.

13. Kilitlenme ve Açlık

Hoca dersi kilitlenme (deadlock) ve açlığa (starvation) kısaca değinerek bitirdi. Bu konular için ayrı bir bölüm vardı: Kilitlenme nasıl ele alınır, oluşması nasıl engellenir, oluştuktan sonra mı bir şey yapılmalı, yoksa başka bir yöntem mi bulunmalı; hepsi orada ayrıca konuşulacaktı.

13.1 Açlık

Açlık, bir sürecin ihtiyaç duyduğu kaynağa hiçbir koşulda erişememesi, çok uzun süre bloklu kalmasıdır. Semaforda konuşursak semafora erişememesidir. Aynı şey CPU için de söylenebilir: Öncelik tabanlı zamanlama yapıyorsunuz, bir sürecin önceliği çok düşük. Hep yüksek öncelikli işler geldiği için ona bir türlü sıra gelmez. Buna bir çözüm bulmak gerekir.

13.2 Kilitlenme

Kilitlenme, iki ya da daha fazla sürecin (hocaya göre bir ya da daha fazla da denebilir), beklediği olayın, yine bekleme durumunda olan ve başka bir olayı bekleyen bir süreç tarafından gerçekleştirilmesini beklemesidir. Hoca bu tanımın karışık gelebileceğini söyleyip şekil üzerinden anlattı.

S ve Q adlı iki semafor vardır; ikisi de 1 ile başlatılır. P_0 ve P_1 süreçleri şöyledir:

```
/* P0 */          /* P1 */
wait(S);          wait(Q);
wait(Q);          wait(S);
...               ...
signal(S);        signal(Q);
signal(Q);        signal(S);
```

P_0 tek başına çalışsa sorun yoktur. `wait(S)` ile S 0 olur, `wait(Q)` ile Q 0 olur. İkisi de 1 olduğu için beklemeden geçer, işini yapar, `signal` ile değerleri tekrar 1 yapar. P_1 tek başına çalıştığında da benzer şekilde önce Q, sonra S alınır, iş yapılır, çıkılır.

Sorun, iki süreç birlikte çalışınca ortaya çıkar:

- **Adım 1:** P_0 `wait(S)` yapar; $S = 0$. Sonra kesilir.
- **Adım 2:** P_1 çalışır, `wait(Q)` yapar; $Q = 0$.
- **Adım 3:** P_0 `wait(Q)` yapmaya çalışır; Q 0 olduğu için bekler.
- **Adım 4:** P_1 `wait(S)` yapmaya çalışır; S 0 olduğu için bekler.

İki süreç birbirini içinden çıkamayacakları bir kilide sokmuştur. P_0, P_1'in tuttuğu Q'yu; P_1 de P_0'ın tuttuğu S'yi bekler.

> **Ders sonrası not:** Bu not hazırlanırken senaryo POSIX semaforlarıyla, her `wait`'in arasına gecikme konarak çalıştırıldı. Zaman aşımlı bekleme (`sem_timedwait`) kullanılmasaydı iki iş parçacığı da sonsuza kadar bekleyecekti; zaman aşımı kilitlenmeyi gösterdi.

Hocanın vurgusu: İşletim sistemi size senkronizasyon araçlarını sağlar: semafor ve üzerindeki `wait`, `signal` işlemleri. Bunları nasıl kullanacağınız size kalmıştır. Yanlış kullanırsanız açlık ya da kilitlenme oluşabilir. Ayrıca tek başına temiz ve düzgün çalışan kod, çok süreçli ya da çok iş parçacıklı olduğunda düzgün çalışmayabilir. Bu hataları bulmak, tespit etmek ve düzeltmek (debugging) her zaman daha zordur.

13.3 Olası çözümler

Hoca açlık için olası çözümlerden söz etti. Yalnız zamanlama açısından bakarsak: Önceliği düşük bir süreci çalıştırmıyorsunuz. Belli bir süre çalışmadığında önceliğini artırabilirsiniz; sonra biraz daha, sonra biraz daha, ta ki süreç CPU'da çalışana kadar.

Ders sonrası not: Bekleyen sürecin önceliğini zamanla artırma yönteminin adı yaşlandırmadır (aging).

Hoca problemi biraz daha karıştırdı: Düşük öncelikli bir süreç zamanlamada yer bulamıyor, ama elinde yüksek öncelikli süreçlerin ihtiyaç duyduğu bir kilit var. Bu durumda tuttuğu kilidin önceliğini, bu süreci zamanlamada öne çekecek şekilde kullanabilirsiniz. Hoca bunu "priority inversion yaparsınız, inherit etmesini sağlarsınız" diye ifade etti.

Ders sonrası not: Öncelik tersinmesi (priority inversion) çözümün değil, sorunun adıdır: Düşük öncelikli süreç, yüksek öncelikli sürecin beklediği kilidi tutar ve yüksek öncelikli süreç fiilen düşük öncelikliye bağımlı kalır. Hocanın tarif ettiği çözüm öncelik kalıtımıdır (priority inheritance): Kilidi tutan süreç, kilidi bırakana kadar bekleyen sürecin yüksek önceliğini geçici olarak alır.

Hoca bunların ayrıntılarının kilitlenme ve açlık konusunda ayrıca konuşulacağını söyledi. Klasik senkronizasyon problemlerine bu hafta geçmedi; yarım kalacağı için bir sonraki derse bıraktı.

Örnek Sorular

Bu derste ekrandan kare yoktur. Sorular yalnız derste sözlü sorulanlardır.

Soru 1 (derste sözlü, 00:17:35): Komut yaşam döngüsünün (instruction life cycle) adımları nelerdir?

Cevap (derste verildi): Getir (fetch), çöz (decode), çalıştır (execute). Bir öğrenci doğru cevapladı.

Soru 2 (derste sözlü, 00:19:16): İşletim sistemi kesme güdümlüdür. Kesmenin gelip gelmediği ne zaman ve nasıl kontrol edilir?

Hoca "zamanlayıcı zaman aşımı" cevabını kabul etmedi; bunun kontrol değil, bir kesme örneği olduğunu söyledi. Soru derste cevapsız kaldı.

Çözüm (derste verilmedi): İşlemci kesme isteği hattını donanım olarak her komut döngüsünün sonunda, bir sonraki komutu getirmeden (fetch) önce kontrol eder. Bekleyen ve maskelenmemiş bir kesme varsa, o anki durumu (program sayacı vb.) kaydedip kesme servis yordamına atlar. Bu yüzden kesme, `counter++` gibi tek bir yüksek seviyeli satırı oluşturan makine komutlarının arasına girebilir. Kesmeleri kapatmak da tam bu kontrolü devre dışı bırakmak demektir.

Soru 3 (derste sözlü, 00:22:29): Üretici ile tüketici tampon alanını paylaşıyor. Tampona nasıl erişiyorlar ve bu paylaşım neden sorun değil?

Cevap (derste verildi): Üretici tampona yalnız yazar, tüketici yalnız okur. Okuma değeri değiştirmez ve her seferinde aynı sonucu verir (eşgüçlü, idempotent). Yazan tek taraf olduğu için sorun yoktur. Sorun, iki tarafın da yazdığı `counter`'dadır.

Soru 4 (derste sözlü, 00:36:05): Kritik bölge sorunu süreçler arası geçiş yüzünden çıkıyor. Bu geçiş neyle sağlanır?

Cevap (derste verildi): Kesmeyle (interrupt). İşletim sistemi kesme güdümlüdür. Bu yüzden akla gelen en basit çözüm kesmeleri kapatmaktır; ama modern işletim sisteminde kabul edilemez.

Soru 5 (derste sözlü, 00:53:56): Peterson çözümünde süreç sayısını artırırsanız neden çalışmaz?

> **Cevap (derste verildi):** Her süreç diğer bütün süreçlerin `flag` değerini kontrol etmek zorunda kalır. Kesmenin nerede geleceği bilinmez: Bir kontrol yapıldıktan sonra, diğerleri yapılırken süreç kesilebilir ve ilk kontrolün sonucu değişebilir. Süreç kaldığı yerden eski bilgiyle devam eder ve karşılıklı dışlama bozulur. Bu yüzden donanım desteği gerekir.

Soru 6 (derste sözlü, 01:09:38): `test_and_set` hedefin içeriğini `true` yapıp eski değeri döndürüyor. Bu bizim ne işimize yarar?

> **Cevap (derste verildi):** Eski değeri görebiliriz. Eski değer `false` ise kilit boştu ve artık bizimdir; `true` ise kilit başkasındadır, döngüde beklenir. İşlem atomik olduğu için iki süreç aynı anda "kilit boştu" sonucunu alamaz.

Soru 7 (derste sözlü, 01:20:10): Spinlock işlemciyi boşuna meşgul ediyor. Bunun yerine ne yapılabilir?

> **Cevap (derste verildi):** Öğrencilerden süreci uyutmak (`sleep`) önerildi; hoca bunun çözüm olmadığını, yalnız meşgul beklemenin süresini azalttığını söyledi, çünkü kaynağın ne zaman boşalacağı bilinmez. Doğru yaklaşım, kilidi alamayan süreci bekleme kuyruğuna almak, kilit bırakılınca hazır kuyruğuna geri koymaktır. Bu yaklaşım semaforda kullanılır.

Dersin Toparlaması

Bu derste senkronizasyon ihtiyacının kaynağı görüldü: Veri paylaşan süreçler kesilebilir biçimde çalıştığında, `counter++` gibi tek satır görünen ama birkaç makine komutundan oluşan işlemler iç içe geçer ve sonuç çalıştırma sırasına bağlı hâle gelir. Buna yarış durumu, ortak veriyi değiştiren koda kritik bölge denir. Bir çözüm karşılıklı dışlama, ilerleme ve sınırlı bekleme şartlarını sağlamalıdır. Kesmeleri kapatmak "başı ağrıyanın başını kesmek" gibidir. Sırayla geçiş ve Peterson çözümü yazılımsal yaklaşımlardır; Peterson iki süreçle sınırlıdır. Donanım `test_and_set` ve `compare_and_swap` gibi atomik komutlar sunar. İşletim sistemi bunları geliştiriciye mutex kilidi ve semafor olarak açar. Spinlock meşgul bekleme yapar ve çok işlemcide kısa kritik bölgeler için uygundur. Bekleme kuyruklu semafor işlemciyi boşa harcamaz, ama gerçeklemesi daha karmaşıktır.

Son olarak araçların yanlış kullanımının kilitlenme ve açlığa yol açabileceği görüldü. Hoca bir sonraki derste klasik senkronizasyon problemlerine (üretici-tüketici, yemek yiyen filozoflar vb.) ve ardından monitor yapısına geçileceğini belirtti. Kilitlenme ise ileride ayrı bir bölümde ayrıntılı ele alınacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.